



Intro to React JS

22/04/2026



```
render() {  
  return (  
    <React.Fragment>  
      <div className="py-5">  
        <div className="container">  
          <div className="row">  
            <ProductConsumer>  
              {(value) => {  
                console.log(value)  
              }}  
            </ProductConsumer>  
          </div>  
        </div>  
      </div>  
    </React.Fragment>  
  )  
}
```



Week 2 - Session 4
Full Stack MERN Bootcamp



Why React?



React lets you build UIs from individual pieces called components.

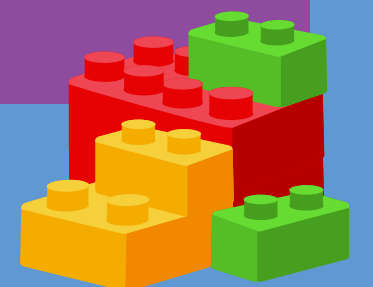
“Used by Meta, Netflix, Shopify—and you!”

Link: <https://react.dev>



React is like LEGO for websites, you build small pieces (components) and snap them together to create big, complex pages.

Create your own React components like Thumbnail, LikeButton, and Video. Then combine them into entire screens.



What is React?



React is a JavaScript library for building fast, interactive user interfaces.

- Use JSX to write HTML-like code inside JavaScript.
- Use a Virtual DOM to update only what's necessary.



Fast → thanks to Virtual DOM



Reusable → components can be reused anywhere



Declarative → you describe what you want, React figures out how to show it



What are Components?



Function name starts with a capital letter



Returns JSX inside return()



Export to use in other files



- Reusable pieces of code that return UI
- Like JavaScript functions but return HTML-like markup (JSX)
- Help break down UI into independent, reusable pieces



<Header/>

Main Content

Footer



What is JSX?



It allows you to write HTML-like syntax inside JavaScript.

Browsers can't read JSX directly, so React compiles it (using tools like Babel) into plain JavaScript.

JSX looks like HTML, but it's just JavaScript. You can write logic directly inside it.



```
const name = "Sara";  
return <h1>Hello, {name}!</h1>;
```



JSX stands for JavaScript XML

JSX is just a friendlier way to write JavaScript UI code



Document Object Model (DOM)

DOM

A tree-like structure that browsers use to represent a web page

JavaScript can add, remove, or modify these nodes to update the page

DOM

```
document
├── html
│   ├── head
│   └── body
│       ├── header
│       ├── main
│       └── footer
```

Virtual DOM

Lightweight copy of the real DOM stored in memory.

CODE CHANGES ↓

Virtual DOM (copy in memory) ↓

React compares old vs new ↓

Figures out minimal changes ↓

Updates ONLY what changed in Real DOM

This makes updates faster and more efficient.



Key Rules



●●● Single Parent Element

Must return one root element (wrap multiple elements in a container or fragment)

●●● Use className for CSS

class is reserved in JavaScript

```
.card { ... }
```

```
<p className="card">.. </p>
```



How it all fits together?



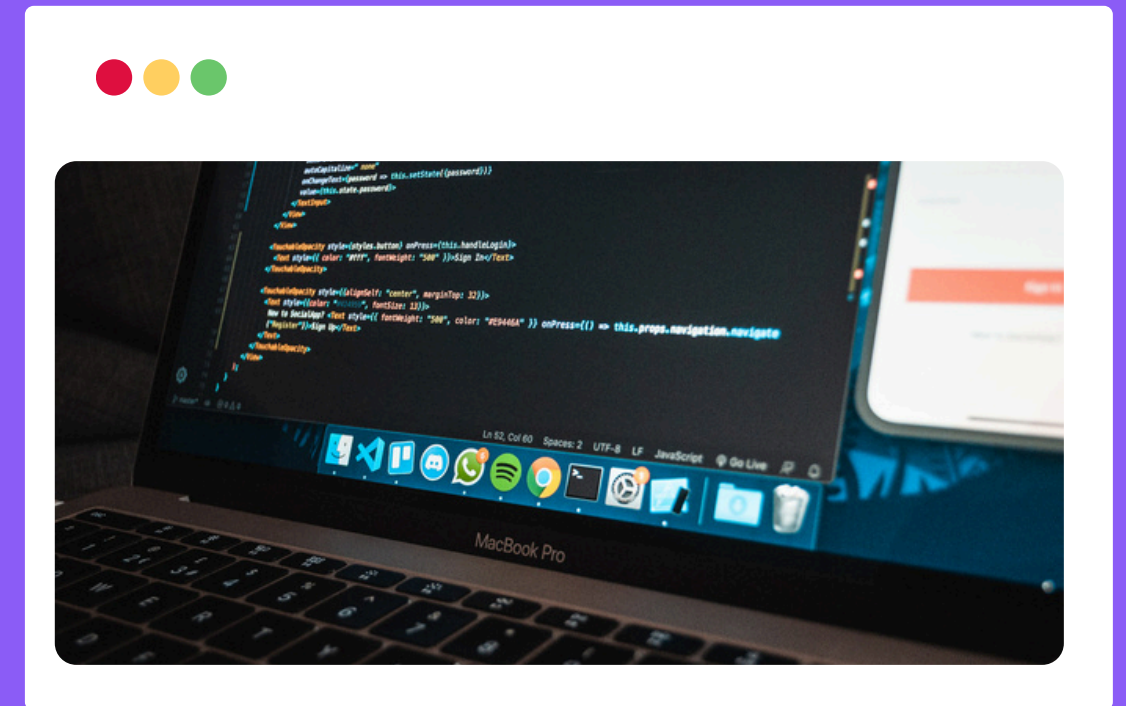
Component →
JSX →
Virtual DOM →
Real DOM



You write components in JSX → React manages the Virtual DOM → Browser shows the real UI



Q&A, Support



Questions? Book your 1:1 or
message me—no question is too
small.

You've got this!

